

Interaction

交互题

上个世纪的 IOI 就已涉及交互题。虽然交互题近年来没有在省选以下的比赛中出现，不过 2019 年里 NOI 系列比赛中连续出现《P5208[WC2019] 君的商店》、《P5473[NOI2019] 君的探险》两道交互题，这可能代表着交互题重新回到 NOI 系列比赛中。

交互题没有很高的前置算法要求，一般也没有严格的时间限制，程序的优秀程度往往仅取决于交互次数限制。所以学习交互题时，建议按照难度循序渐进。要是有意锻炼算法思维而不只是单纯地学习算法，那么完成交互题是很不错的方法。虽然交互题对选手已掌握算法的要求通常较低，但仍建议掌握一定提高和省选算法后再尝试做交互题，因为此时自己的算法思维水平和知识面已经达到了一定水平。基础的交互题介绍可以参考 [OI Wiki 的 题型介绍 - 交互题](#)。

交互题的特殊错误：

- 选手每一次输出后都需要刷新缓冲区，否则会引起 `Idleness limit exceeded` 错误。另外，如果题目含多组数据并且程序可以在未读入所有数据前就知道答案，也仍然要读入所有数据，否则同样会因为读入混乱引起 ILE（可以一次提出多次询问，一次接收所有询问的回答）。同时尽量不要使用快读。
- 如果程序查询次数过多，则在 Codeforces 上会给出 `Wrong Answer` 的评测结果（不过评测系统会说明 `Wrong Answer` 的原因），而 UVa 会给出 `Protocol Limit Exceeded (PLE)` 的评测结果。
- 如果程序交互格式错误，UVa 会给出 `Protocol Violation (PV)` 的评测结果。

由于交互题输入输出较为繁琐，所以建议分别封装输入和输出函数。

比赛时如果出题人给出了 `grader` 头文件（用于 `grader` 交互题的调试）或者 `checker` 程序（用于 `stdio` 交互题的调试），则交互题的调试比较简单，因为交互题的对拍会比普通题目的对拍困难很多。没有 `testlib.h` 的情况下。交互细节较多的题目的 `stdio` 交互库会一般有 3k 代码量，再加上 3k 长度的对拍器，至少需要一小时实现。但是，无论是否有调试程序，调试交互题的代码都往往需要选手模拟与程序的交互过程，因此交互题需要选手能设计出高质量的程序，尽量保证一遍做对，同时拥有较强的静态查错能力。

例题：

- [CF679A Bear and Prime 100](#)
- [CF843B Interactive LowerBound](#)
- [UOJ206\[APIO2016\]Gap](#)
- [CF750F New Year and Finding Roots](#)
- [UVa12731 太空站之谜 Mysterious Space Station](#)

CF679A Bear and Prime 100

每个质数都有且只有两个因数，所以直接枚举要猜的数的因数。由于限制最多询问 20 次，并且对于较大的数（如 92）尝试分解质因数时发现需要最多枚举到 的质数。所以我们先筛出 50 以内的质数，每次把所有这些数都询问一遍。

由于本题对拍比较容易，可以直接把值域内的数都尝试一遍。我们会发现程序无法有效处理质数的平方。所以我们要把 2,3,5,7 的平方 4,9,25,49 都放进去，总共 19 个数字，符合题意。

► 参考代码

CF843B Interactive LowerBound

链表最多有 个元素，但我们只能询问 次，并且只能获取元素的最后一个元素，所以普通的遍历整个链表的方法不可用。直接设法逼近目标元素的位置只有一种方法：随机撒点。

对于的情况直接枚举，时，我们直接撒 1000 个点，这时这些点之间的期望距离很小，我们可以直接从小于 的最大值开始向后遍历，可以证明在到达下一个点之前我们就已得到答案。遍历的过程中一旦找到大于等于 的元素，就可以直接推出。

虽然整体思路简单，但实际情况下，如果没有学习过模拟退火等非完美随机算法，思考起来很可能会困难一些。

同时由于 Codeforces 具有 hack 机制，很多人会刻意卡掉没有初始化随机种子的代码，所以在 `random_shuffle()` 函数前需要 `srand((size_t)new char)`。

► 参考代码

UOJ206[APIO2016]Gap

分两个子任务讨论：

1. 查询次数限制。

我们考虑第一次查询。因为我们一开始不知道任何数，所以我们需要询问范围，获得最大最小值。

由于查询次数限制刚好为，所以考虑怎么每一次都能获取之前没有获取过的值，这样能大概在次数范围内获取序列内的所有数。方法也很简单：每次查询后，设获得的值为，则下一次查询。

2. 询问区间大小限制。

由于题目要求询问区间内的数的数量之和不能超过，所以考虑最小化询问区间。上面的方法不再可用，因为其询问区间内的数数量之和规模为。我们可以考虑二分值域，但这种方法并不可靠，最坏可能会被卡到。所以我们需要更有效的划分值域的方法，避免查询区间内的点重复查询，浪费机会。

考虑到答案不会小于，所以我们可以考虑按这个值划分值域，设初始为 0，初始为上述值，每次询问并且更新，之后再以为步长让自增。

不过这种方法也不能很好地适用于子任务 1，因为最坏可能很多询问的值域内一个数都没有。

► 参考代码

CF750F New Year and Finding Roots

看到，询问次数 的严格要求，我们需要非常严格地最大化利用访问获得的信息。

时可以直接暴力枚举。然而时需要很高效的遍历算法。

随机撒点不是好方法，因为随机撒点无法确定自己是否足够接近根节点了，并且单纯随机撒点，至少有一次碰到根节点的概率为，即使排除重复撒点的情况后，碰到根节点的概率仍然非常小。

由于，并且我们并不知道哪一边更接近根节点，所以我们考虑最坏的情况，即如果时，前两次我们的遍历方向都是远离根节点的，第三次遍历方向是接近根节点的。所以我们必须往三个方向都遍历。

考虑 bfs 和 dfs 两种遍历方法。由于 bfs 搜索树可能很大，所以我们优先考虑 dfs。当然，如果我们知道当前的深度，并且当前深度小到深度范围内的搜索树规模小于等于剩余次数，我们就可以直接 bfs。

知道当前节点的深度，以及当前遍历的方向会获得很大优势。然而知道当前在往根节点还是在往叶子节点遍历是非常困难的事情。如果使用 dfs，只有当遍历到根节点（）或者叶子节点（）时才知道当前方向。所以我们需要尽可能知道当前节点深度，并且不能采用类似迭代加深搜索的方法，遍历中途停下来。

考虑随机一个初始节点，从初始节点出发可能碰到上面的最坏情况。

如果，我们就可以直接知道当前节点的深度。

如果，那当前节点即根节点。

如果，我们直接考虑往三个方向 dfs。考虑到其中两个方向是直接往叶子节点的方向，遍历路径长度相同；另一个方向是往根节点的方向，不过可能中途不小心往叶子节点的方向走了，遍历路径长度会较大。此时我们就可以计算出当前节点的深度。

当或者时，我们需要考虑较长的遍历路径。我们可以知道路径上深度最小的点（必定比初始节点深度小）。如果我们为访问过的节点打标记，不再遍历，此时从该节点开始就只有一条遍历路径。虽然这条路径可能还是会走向叶子节点，但是这条路径上同样必然存在深度比起点小的节点，我们就可以从这个节点开始继续重复上面的步骤。

当然，我们考虑的最坏情况时（每次只往根节点走一步，就直接往叶子节点走），会发现如果只 dfs，最坏需要次询问。不过我们已经知道初始节点的深度，所以我们可以算出所有已遍历节点的深度，并且根据我们开始时对 bfs 的讨论，判断是否可以从深度最小的点直接 bfs。

这时，我们可以算出最坏需要 17 次。所以我们考虑从搜索树上去掉一个节点（根据 dfs 只能盲目遍历的性质，我们考虑 bfs）：即当进行深度为的 bfs 时，搜索树节点最坏有个，可能需要次询问才能确定哪个节点的邻居恰有 2 个。不过我们如果已经对其中个节点询问后，可以知道最后一个节点肯定是根节点。

此时最坏情况下的最优解为：时，从叶子节点 dfs，每次都是只往根节点走一步，就直接往叶子节点走，询问 10 次后，当前已知最小深度的节点深度为 4，由于已知其父亲，直接从其父亲开始 bfs（搜索树深度为 3，节点数为）。在 bfs 时询问了次后，确定 bfs 搜索树上最后一个节点为根节点。

此时我们的算法可以刚好卡到最坏 16 次。

► 参考代码

UVa12731 太空站之谜 Mysterious Space Station

由于唯一的反馈是移动时是否撞墙，所以我们应该考虑在机器人不走丢的情况下，尽量接近墙边走路，这样有几个好处：

- 靠近墙边走路时，很容易知道自己会不会撞墙，获取到尽量多的信息。
- 墙边都是不会出现传送门的格子，可以避免机器人走丢。

所以，我们如果已知机器人可能在墙边的某个位置，要确定机器人是不是真的在这个位置，就可以通过「[单手扶墙法](#)」确定自己是不是真的在这个位置。根据拓扑学原理，在两边都是墙的迷宫中，如果从入口进入，并且总是用一只手扶着同一边墙，就可以保证找到出口。由于本题中的墙是闭合的，所以只需要沿着墙边的道路走，就可以保证可以回到原点而不会撞墙。另外，由于墙边的道路是地图上的最大闭合回路，所以实际代码中并不需要特意撞墙以保证机器人在墙边，可以使用标记在地图中标明墙边道路。而且一旦撞了墙，就需要赶快沿着原路返回，可以在避免机器人走丢的同时减少步数。

由上，可以推断出确定机器人是否在特定格子的试错法：将机器人在不走到未知格子或已知传送门的情况下走到墙边的道路上，然后绕着墙边道路走一圈。这个过程中如果没有撞墙，就可以确定机器人确实是在特定格子。

我们可以采用上面的方法，一开始标出图中所有未知格子，然后从上到下，从左到右依次判断每个未知格子是否是传送门。可以先走到未知格子上方，然后向下、向左走。再用上面的方法判断机器人是不是在未知格子的左侧。如果不是，说明机器人不在应该在的位置，即未知格子是传送门。

找出未知格子后就需要判断 $2k$ 个未知格子的配对关系，实际方法也很简单：只需要暴力配对就可以了。由于，所以最多只需要次试错法。作为对比，判断图中全部未知格子的情况最多需要次试错法。

由于目前下面这一份代码只能通过 UOJ 的镜像题：[#247. 【Rujia Liu's Present 7】Mysterious Space Station](#)，而无法通过 UVa 原题。修改了 UOJ 上刘汝佳的标程后还是无法通过，并且暂时无法联系到刘汝佳。所以下面的代码以 UOJ 为准。

不过刘汝佳的标程质量还是比下面这份代码质量高很多的，可以在 UOJ 上查看到[通过了 UOJ 镜像题的标程](#)。同一份

数据下，标程使用的移动次数非常少。

► [参考代码](#)

习题

- [刘汝佳的交互题专场比赛 Rujia Liu's Present 7 质量非常高，推荐一做。](#)
- [P5473\[NOI2019\]I 君的探险](#)
- [P5208\[WC2019\]I 君的商店](#)

参考资料与拓展阅读

- [用 Linux 管道实现 online judge 的交互题功能](#)
-

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[StudyingFather](#), [countercurrent-time](#), [billchenchina](#), [CCXXI](#), [Enter-tainer](#), [gi-b716](#), [lr1d](#), [ksyx](#), [NachtgeistW](#), [ouuan](#), [Tiphereth-A](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用